6. Metaclasses and Attributes

Metaclasses are often mentioned in lists of Python's features, but few understand what they accomplish in practice. The name *metaclass* vaguely implies a concept above and beyond a class. Simply put, metaclasses let you intercept Python's `class` statement and provide special behavior each time a class is defined.

Similarly mysterious and powerful are Python's built-in features for dynamically customizing attribute accesses. Along with Python's object-oriented constructs, these facilities provide wonderful tools to ease the transition from simple classes to complex ones.

However, with these powers come many pitfalls. Dynamic attributes enable you to override objects and cause unexpected side effects. Metaclasses can create extremely bizarre behaviors that are unapproachable to newcomers. It's important that you follow the *rule of least surprise* and only use these mechanisms to implement well-understood idioms.

ITEM 44: USE PLAIN ATTRIBUTES INSTEAD OF SETTER AND GETTER METHODS

Programmers coming to Python from other languages may naturally try to implement explicit getter and setter methods in their classes:

```
class OldResistor:
    def __init__(self, ohms):
        self._ohms = ohms

    def get_ohms(self):
        return self._ohms

    def set_ohms(self, ohms):
        self._ohms = ohms
```

Using these setters and getters is simple, but it's not Pythonic:

```
r0 = OldResistor(50e3)
print('Before:', r0.get_ohms())
r0.set_ohms(10e3)
print('After: ', r0.get_ohms())

>>>
```

```
Before: 50000.0
After:  10000.0
```

Such methods are especially clumsy for operations like incrementing in place:

```
r0.set_ohms(r0.get_ohms() - 4e3)
assert r0.get_ohms() == 6e3
```

These utility methods do, however, help define the interface for a class, making it easier to encapsulate functionality, validate usage, and define boundaries. Those are important goals when designing a class to ensure that you don't break callers as the class evolves over time.

In Python, however, you never need to implement explicit setter or getter methods. Instead, you should always start your implementations with simple public attributes, as I do here:

```
class Resistor:
    def __init__(self, ohms):
        self.ohms = ohms
        self.voltage = 0
        self.current = 0

r1 = Resistor(50e3)
r1.ohms = 10e3
```

These attributes make operations like incrementing in place natural and clear:

```
r1.ohms += 5e3
```

Later, if I decide I need special behavior when an attribute is set, I can migrate to the `@property` decorator (see Item 26: "Define Function Decorators with `functools.wraps`" for background) and its corresponding `setter` attribute. Here, I define a new subclass of `Resistor` that lets me vary the `current` by assigning the `voltage` property. Note that in order for this code to work properly, the names of both the setter and the getter methods must match the intended property name:

Click here to view code image

```
class VoltageResistance(Resistor):
    def __init__(self, ohms):
        super().__init__(ohms)
        self._voltage = 0
```

```
    @property
    def voltage(self):
        return self._voltage

    @voltage.setter
    def voltage(self, voltage):
        self._voltage = voltage
        self.current = self._voltage / self.ohms
```

Now, assigning the `voltage` property will run the `voltage` setter method, which in turn will update the `current` attribute of the object to match:

Click here to view code image

```
r2 = VoltageResistance(1e3)
print(f'Before: {r2.current:.2f} amps')
r2.voltage = 10
print(f'After:  {r2.current:.2f} amps')

>>>
Before: 0.00 amps
After:  0.01 amps
```

Specifying a `setter` on a property also enables me to perform type checking and validation on values passed to the class. Here, I define a class that ensures all resistance values are above zero ohms:

Click here to view code image

```
class BoundedResistance(Resistor):
    def __init__(self, ohms):
        super().__init__(ohms)

    @property
    def ohms(self):
        return self._ohms

    @ohms.setter
    def ohms(self, ohms):
        if ohms <= 0:
            raise ValueError(f'ohms must be > 0; got {ohms}')
        self._ohms = ohms
```

Assigning an invalid resistance to the attribute now raises an exception:

```
r3 = BoundedResistance(1e3)
r3.ohms = 0
>>>
Traceback ...
ValueError: ohms must be > 0; got 0
```

An exception is also raised if I pass an invalid value to the constructor:

Click here to view code image

```
BoundedResistance(-5)

>>>
Traceback ...
ValueError: ohms must be > 0; got -5
```

This happens because `BoundedResistance.__init__` calls `Resistor.__init__`, which assigns `self.ohms = -5`. That assignment causes the `@ohms.setter` method from `BoundedResistance` to be called, and it immediately runs the validation code before object construction has completed.

I can even use `@property` to make attributes from parent classes immutable:

Click here to view code image

```
class FixedResistance(Resistor):
    def __init__(self, ohms):
        super().__init__(ohms)

    @property
    def ohms(self):
        return self._ohms

    @ohms.setter
    def ohms(self, ohms):
        if hasattr(self, '_ohms'):
            raise AttributeError("Ohms is immutable")
        self._ohms = ohms
```

Trying to assign to the property after construction raises an exception:

```
r4 = FixedResistance(1e3)
r4.ohms = 2e3
```

```
>>>
Traceback ...
AttributeError: Ohms is immutable
```

When you use `@property` methods to implement setters and getters, be sure that the behavior you implement is not surprising. For example, don't set other attributes in getter property methods:

Click here to view code image

```
class MysteriousResistor(Resistor):
    @property
    def ohms(self):
        self.voltage = self._ohms * self.current
        return self._ohms

    @ohms.setter
    def ohms(self, ohms):
        self._ohms = ohms
```

Setting other attributes in getter property methods leads to extremely bizarre behavior:

```
r7 = MysteriousResistor(10)
r7.current = 0.01
print(f'Before: {r7.voltage:.2f}')
r7.ohms
print(f'After:  {r7.voltage:.2f}')

>>>
Before: 0.00
After:  0.10
```

The best policy is to modify only related object state in `@property.setter` methods. Be sure to also avoid any other side effects that the caller may not expect beyond the object, such as importing modules dynamically, running slow helper functions, doing I/O, or making expensive database queries. Users of a class will expect its attributes to be like any other Python object: quick and easy. Use normal methods to do anything more complex or slow.

The biggest shortcoming of `@property` is that the methods for an attribute can only be shared by subclasses. Unrelated classes can't share the same implementation. However, Python also supports *descriptors* (see Item 46: "Use Descriptors for Reusable `@property` Methods") that enable reusable property logic and many other use cases.

Things to Remember

✦ Define new class interfaces using simple public attributes and avoid defining setter and getter methods.

✦ Use `@property` to define special behavior when attributes are accessed on your objects, if necessary.

✦ Follow the rule of least surprise and avoid odd side effects in your `@property` methods.

✦ Ensure that `@property` methods are fast; for slow or complex work—especially involving I/O or causing side effects—use normal methods instead.

ITEM 45: CONSIDER @PROPERTY INSTEAD OF REFACTORING ATTRIBUTES

The built-in `@property` decorator makes it easy for simple accesses of an instance's attributes to act smarter (see Item 44: "Use Plain Attributes Instead of Setter and Getter Methods"). One advanced but common use of `@property` is transitioning what was once a simple numerical attribute into an on-the-fly calculation. This is extremely helpful because it lets you migrate all existing usage of a class to have new behaviors without requiring any of the call sites to be rewritten (which is especially important if there's calling code that you don't control). `@property` also provides an important stopgap for improving interfaces over time.

For example, say that I want to implement a leaky bucket quota using plain Python objects. Here, the `Bucket` class represents how much quota remains and the duration for which the quota will be available:

Click here to view code image

```
from datetime import datetime, timedelta

class Bucket:
    def __init__(self, period):
        self.period_delta = timedelta(seconds=period)
        self.reset_time = datetime.now()
        self.quota = 0

    def __repr__(self):
        return f'Bucket(quota={self.quota})'
```

The leaky bucket algorithm works by ensuring that, whenever the bucket is filled, the amount of quota

does not carry over from one period to the next:

Click here to view code image

```
def fill(bucket, amount):
    now = datetime.now()
    if (now - bucket.reset_time) > bucket.period_delta:
        bucket.quota = 0
        bucket.reset_time = now
    bucket.quota += amount
```

Each time a quota consumer wants to do something, it must first ensure that it can deduct the amount of quota it needs to use:

Click here to view code image

```
def deduct(bucket, amount):
    now = datetime.now()
    if (now - bucket.reset_time) > bucket.period_delta:
        return False  # Bucket hasn't been filled this period
    if bucket.quota - amount < 0:
        return False  # Bucket was filled, but not enough

bucket.quota -= amount
return True       # Bucket had enough, quota consumed
```

To use this class, first I fill the bucket up:

```
bucket = Bucket(60)
fill(bucket, 100)
print(bucket)

>>>
Bucket(quota=100)
```

Then, I deduct the quota that I need:

Click here to view code image

```
if deduct(bucket, 99):
    print('Had 99 quota')
else:
    print('Not enough for 99 quota')
```

```
print(bucket)

>>>
Had 99 quota
Bucket(quota=1)
```

Eventually, I'm prevented from making progress because I try to deduct more quota than is available. In this case, the bucket's quota level remains unchanged:

```
if deduct(bucket, 3):
    print('Had 3 quota')
else:
    print('Not enough for 3 quota')
print(bucket)

>>>
Not enough for 3 quota
Bucket(quota=1)
```

The problem with this implementation is that I never know what quota level the bucket started with. The quota is deducted over the course of the period until it reaches zero. At that point, `deduct` will always return `False` until the bucket is refilled. When that happens, it would be useful to know whether callers to `deduct` are being blocked because the `Bucket` ran out of quota or because the `Bucket` never had quota during this period in the first place.

To fix this, I can change the class to keep track of the `max_quota` issued in the period and the `quota_consumed` in the period:

Click here to view code image

```
class NewBucket:
    def __init__(self, period):
        self.period_delta = timedelta(seconds=period)
        self.reset_time = datetime.now()
        self.max_quota = 0
        self.quota_consumed = 0

    def __repr__(self):
        return (f'NewBucket(max_quota={self.max_quota}, '
                f'quota_consumed={self.quota_consumed})')
```

To match the previous interface of the original `Bucket` class, I use a `@property` method to compute the current level of quota on-the-fly using these new attributes:

Click here to view code image

```
    @property
    def quota(self):
        return self.max_quota - self.quota_consumed
```

When the `quota` attribute is assigned, I take special action to be compatible with the current usage of the class by the `fill` and `deduct` functions:

Click here to view code image

```
    @quota.setter
    def quota(self, amount):
        delta = self.max_quota - amount
        if amount == 0:
            # Quota being reset for a new period
            self.quota_consumed = 0
            self.max_quota = 0
        elif delta < 0:
            # Quota being filled for the new period
            assert self.quota_consumed == 0
            self.max_quota = amount
       else:
           # Quota being consumed during the period
           assert self.max_quota >= self.quota_consumed
           self.quota_consumed += delta
```

Rerunning the demo code from above produces the same results:

Click here to view code image

```
bucket = NewBucket(60)
print('Initial', bucket)
fill(bucket, 100)
print('Filled', bucket)

if deduct(bucket, 99):
    print('Had 99 quota')
else:
    print('Not enough for 99 quota')
print('Now', bucket)

if deduct(bucket, 3):
    print('Had 3 quota')
else:
```

```
    print('Not enough for 3 quota')

print('Still', bucket)

>>>
Initial NewBucket(max_quota=0, quota_consumed=0)
Filled NewBucket(max_quota=100, quota_consumed=0)
Had 99 quota
Now NewBucket(max_quota=100, quota_consumed=99)
Not enough for 3 quota
Still NewBucket(max_quota=100, quota_consumed=99)
```

The best part is that the code using `Bucket.quota` doesn't have to change or know that the class has changed. New usage of `Bucket` can do the right thing and access `max_quota` and `quota_consumed` directly.

I especially like `@property` because it lets you make incremental progress toward a better data model over time. Reading the `Bucket` example above, you may have thought that `fill` and `deduct` should have been implemented as instance methods in the first place. Although you're probably right (see Item 37: "Compose Classes Instead of Nesting Many Levels of Built-in Types"), in practice there are many situations in which objects start with poorly defined interfaces or act as dumb data containers. This happens when code grows over time, scope increases, multiple authors contribute without anyone considering long-term hygiene, and so on.

`@property` is a tool to help you address problems you'll come across in real-world code. Don't overuse it. When you find yourself repeatedly extending `@property` methods, it's probably time to refactor your class instead of further paving over your code's poor design.

Things to Remember

✦ Use `@property` to give existing instance attributes new functionality.

✦ Make incremental progress toward better data models by using `@property`.

✦ Consider refactoring a class and all call sites when you find yourself using `@property` too heavily.